Práctica 8. Herencia, segunda parte

Modelos de Programación Orientada a Objetos

Integrantes: Hernández Nieves Emily
Sanchez Mendez Citlali Itzel
Números de cuenta: 426058256
426058988
Grupo: 6

En esta práctica continuamos trabajando con la herencia, esta vez con el objetivo de desarrollar del programa de la estación de clasificación, en el que se utilizan sensores y actuadores para representar diferentes acciones. Durante el trabajo se crearon clases, se aprovecharon características de otras clases y se probaron los métodos para comprobar que el programa funcionara como se esperaba.
También revisamos cómo se relacionan las clases entre sí y para qué sirven los modificadores de acceso. Esto nos ayudó a complementar aprendizaje, pues entendimos cómo organizar un programa y cómo evitar que se utilicen ciertos métodos desde lugares donde no deberían estar disponibles.

Desarrollo de la práctica

Primero organizamos el programa en diferentes paquetes para mantener las clases separadas según su función. En el paquete dispositivos se encuentra la clase principal de la que parten los sensores y actuadores. Después, en los paquetes sensores y actuadores, se encuentran las clases que representan los dispositivos específicos, como el sensor de peso, el sensor de temperatura, la banda transportadora, el semáforo y la alarma.
Por último, en el paquete estacion se encuentra la clase que reúne los dispositivos y controla los ciclos de trabajo. Esta organización permite que cada clase tenga una función más clara y que el código sea más fácil de revisar y modificar.

El diagrama UML muestra cómo se organiza el programa y cómo se relacionan las clases. Las flechas de herencia indican qué clases parten de otras, mientras que los rombos representan relaciones de composición o agregación. También se muestran los atributos y métodos principales de cada clase, junto con las cantidades de objetos que pueden relacionarse. Con este diagrama es más fácil entender la estructura del programa antes de revisar el código.

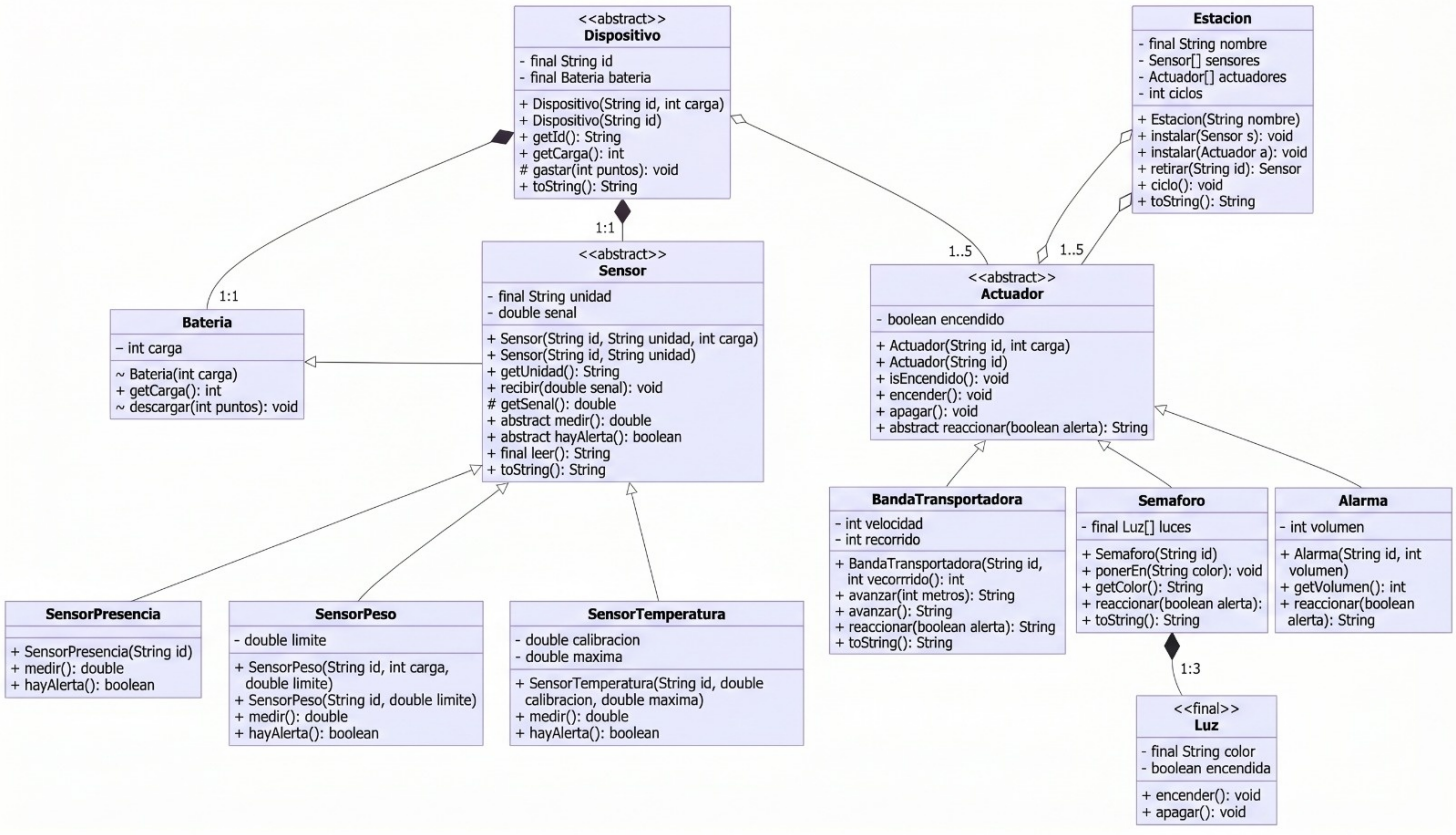


Figura 1. Diagrama UML

En el programa utilizamos la herencia para aprovechar características que varias clases tienen en común. Por ejemplo, los sensores y los actuadores heredan de Dispositivo, por lo que pueden utilizar los atributos y métodos que se definieron en esa clase. Esto evita repetir código y permite que las clases específicas se concentren en las funciones que les corresponden.
También utilizamos super(...) para llamar al constructor de la clase padre y this(...) cuando es necesario llamar a otro constructor de la misma clase. De esta manera, la inicialización de los objetos queda organizada y se aprovecha el código que ya está definido.

Otra parte de la práctica fue trabajar con las relaciones entre objetos. Algunos dispositivos tienen componentes propios, como ocurre con la batería, mientras que la estación reúne sensores y actuadores para utilizarlos durante su funcionamiento. Aunque las relaciones pueden parecer similares, no todas significan lo mismo, por lo que es importante distinguir entre composición y agregación.
Además, revisamos cómo se instalan y retiran los sensores y cómo se conserva su estado cuando vuelven a utilizarse. Esto se puede observar con el sensor de peso S2, que se retira durante la ejecución y después vuelve a instalarse.

Para comprobar el funcionamiento, compilamos los archivos Java y ejecutamos la clase Main. Durante los ciclos se muestran las mediciones de los sensores y las acciones de los actuadores. Cuando un valor supera el límite establecido, aparece una alerta y los actuadores responden de acuerdo con lo programado.
También ejecutamos la clase PruebaTemperatura para comprobar el funcionamiento del sensor de temperatura. En esta prueba se observa que el sensor puede medir una temperatura normal y después mostrar una alerta cuando el valor supera el máximo permitido.

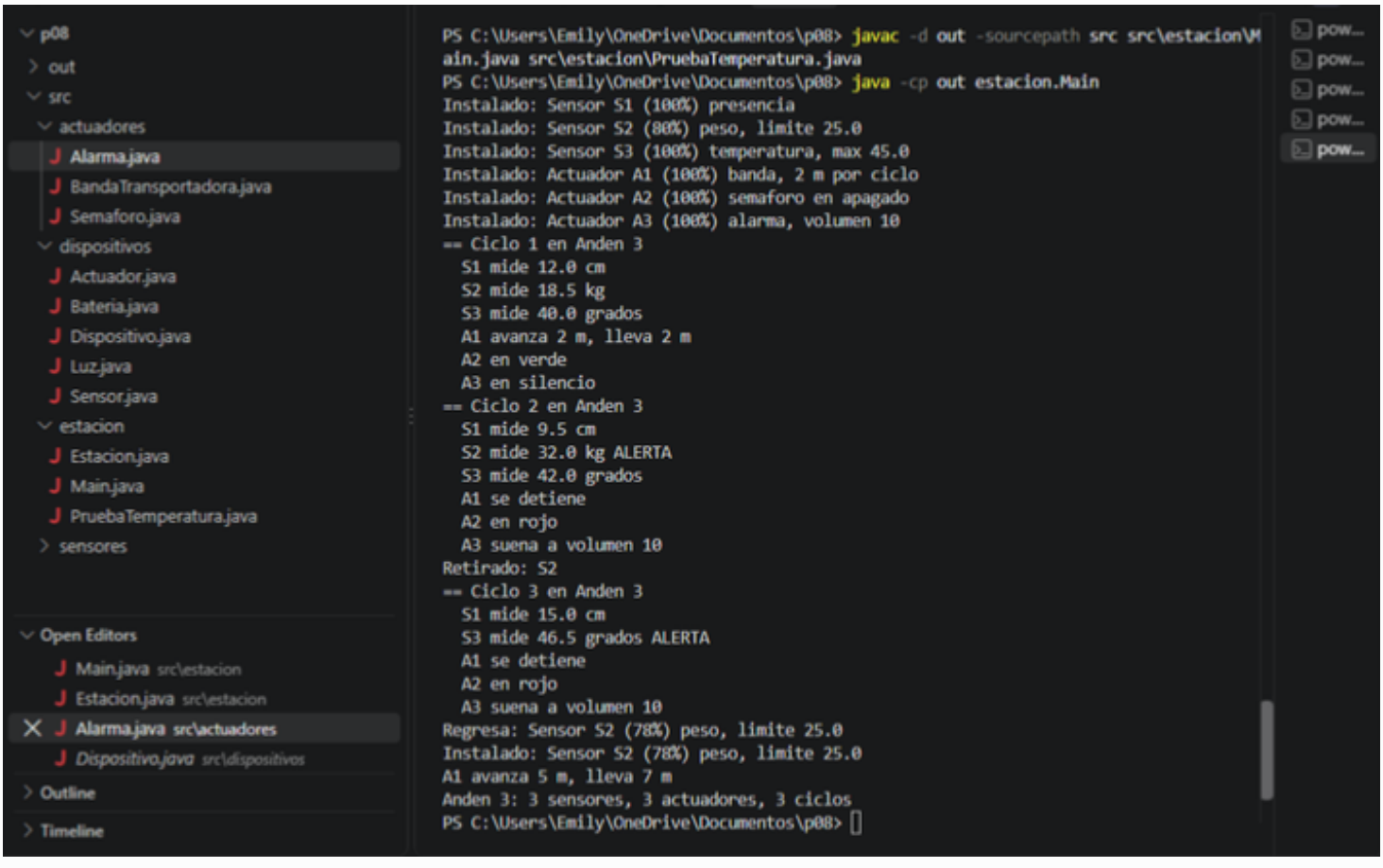


Figura 2. Ejecución de Main.

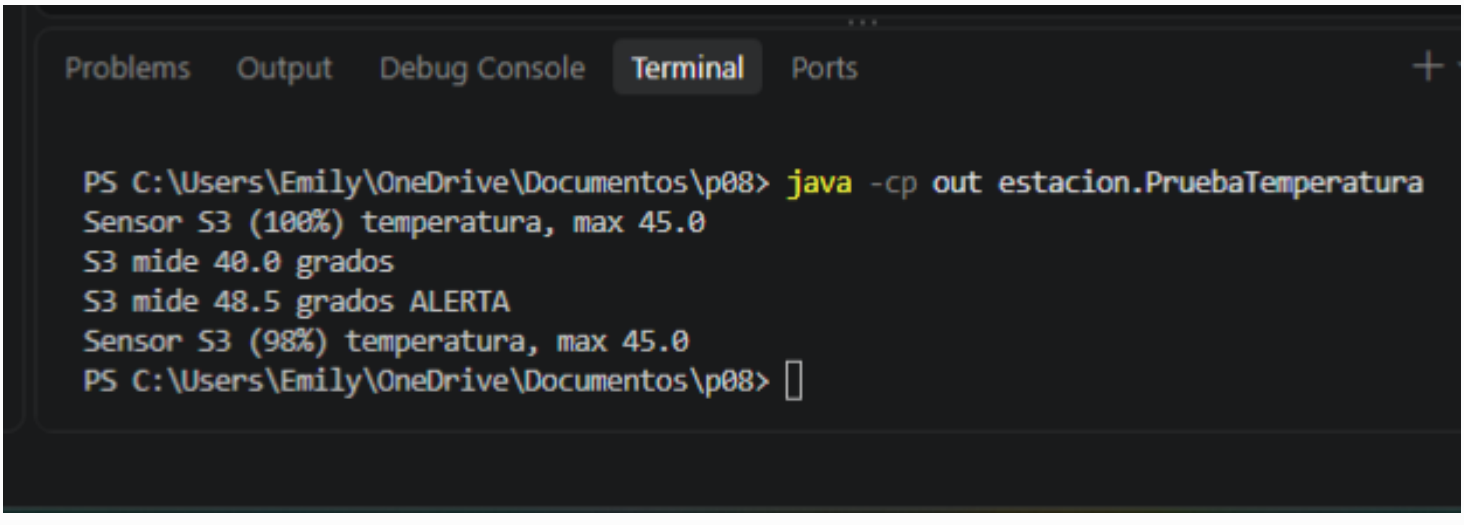


Figura 3. Ejecución de PruebaTemperatura.

Cuestionario

1. ¿Cuál es la diferencia entre sobrecarga y sobrescritura? Incluye un ejemplo de cada una.

La sobrecarga ocurre cuando una clase tiene varios métodos con el mismo nombre, pero con diferentes parámetros. Esto permite utilizar un mismo nombre para realizar una acción de distintas maneras, dependiendo de los datos que se reciban. Por otro lado, la sobrescritura ocurre cuando una clase hija vuelve a definir un método que ya existe en su clase padre, adaptándolo a su propio funcionamiento.
En el programa, la sobrecarga se puede identificar en los constructores que permiten crear un objeto de diferentes maneras. La sobrescritura se utiliza cuando una clase específica define su propia versión de un método heredado, para que el comportamiento corresponda al tipo de dispositivo.

2. Explica la diferencia entre composición y agregación. Utiliza como ejemplo la batería y los sensores de la estación.

La composición representa una relación más fuerte entre dos objetos, porque uno contiene una parte que forma parte de su estructura. En nuestro programa, la relación entre un dispositivo y su batería representa esta idea. En cambio, la agregación permite que un objeto reúna otros objetos sin que estos tengan que existir únicamente como parte de él. Esto ocurre con la estación, que reúne sensores y actuadores para realizar sus ciclos.
Un ejemplo es el sensor de peso S2: cuando se retira de la estación, deja de participar en los ciclos, pero el objeto puede conservarse y volver a instalarse después. Esto permite observar que retirar un sensor de la estación no significa necesariamente eliminarlo.

3. ¿Por qué SensorPeso puede utilizar el método gastar(int), pero Main no puede hacerlo directamente?

Al agregar temporalmente la instrucción bascula.gastar(5); en Main, el compilador muestra un error porque el método gastar(int) tiene acceso protected en la clase Dispositivo. Esto significa que no está disponible para cualquier clase que intente utilizarlo desde cualquier lugar.
SensorPeso puede acceder al método porque es una clase hija de Dispositivo, mientras que Main no hereda de esa clase. Por eso, aunque el objeto bascula exista, Main no puede llamar directamente a ese método desde su código. Después de comprobar el error, quitamos la instrucción para que el programa quedara listo para la entrega.

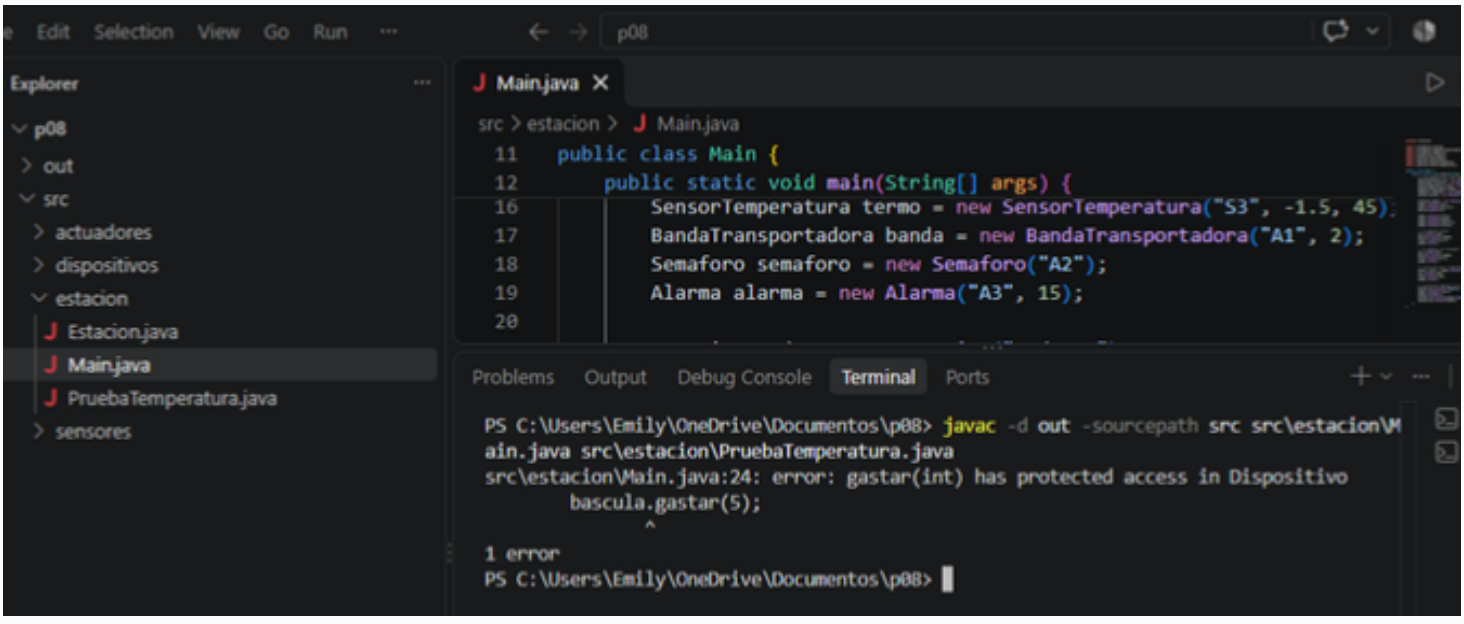


Figura 4. error de compilación al utilizar bascula.gastar

Conclusión

Con esta práctica aprendimos a utilizar la herencia para organizar mejor las clases y evitar el estar repitiendo código. También vimos cómo funcionan los constructores, las relaciones entre objetos y los modificadores de acceso. Las pruebas nos permitieron comprobar que los sensores realizan sus mediciones, que las alertas aparecen cuando corresponde y que los actuadores responden de acuerdo con los valores obtenidos.
En general, la práctica nos ayudó a entender mejor cómo se pueden organizar distintos objetos dentro de un mismo programa y cómo cada clase puede tener una función específica sin dejar de trabajar junto con las demás.